

01 / DECISION AND CURRENT STATE

A real blockchain is achievable

The application can remain a laboratory test system while its proofs are committed to an actual Hyperledger Fabric ledger.

Current decision: NOT READY for a real-blockchain claim.

Fabric integration code exists, but this machine has no usable Docker/WSL runtime and no generated network identity or channel block. Application defaults still select the file ledger.

What is already implemented

The repository contains two peer organizations, one Raft orderer, the **labrecords** channel configuration, and the **labrecord-contract** smart contract. The Fabric adapter signs proposals, submits transactions and waits for a successful commit status. These are real integration components; their presence is not evidence that the network has run. [R2, R3, R6]

CHECK	OBSERVED RESULT
Node.js	v24.18.0 installed.
WSL / Docker	WSL reports not installed. Docker command not found.
Memory / disk snapshot	About 8 GB RAM total, 1 GB free; 188 GB free disk.
Ledger selection	Current .env and native launcher select file.
Fabric artifacts	Application certificate and channel block absent.
Validation today	16 contract/verification tests pass; static infrastructure validation passes. No live Fabric test run.

Recommended first target

One local Fabric test network: RHU peer + verifier peer + one orderer, with encrypted records stored off-chain. Keep the existing two-organization endorsement policy. A local permissioned network is real blockchain execution even when all containers run on one PC. It does not demonstrate independent hosts or orderer failover. [R3, S3, S4]

Assessment scope: source review, local prerequisite checks, isolated defect reproductions and existing tests. No application code, .env, ledger, accounts or network configuration was changed for this assessment. Memory readings are approximate and time-dependent.

02 / YOUR PREPARATION CHECKLIST

Prepare this machine first

Owner: you. These tasks can be completed while the code changes on pages 3-5 are prepared.

☐ 1. Confirm hardware virtualization and available memory

Check Task Manager > Performance > CPU for Virtualization: Enabled. The firmware/SLAT check was blocked by local read permissions, so it is not verified. Close heavy applications before testing. With 8 GB total RAM and about 1 GB free, start with a small workload; consider 16 GB or a separate test host if the full stack is unstable. The 16 GB figure is a planning recommendation, not a Fabric requirement. [S2]

☐ 2. Install WSL and restart Windows

Run PowerShell as Administrator. Finish the Linux distribution setup after restarting, then verify the installed distribution uses version 2. Microsoft documents the installation and reboot steps. [S1]

```
wsl --install
# Restart Windows and finish distribution setup
wsl --update
wsl --version
wsl -l -v
```

☐ 3. Install and start Docker Desktop

Use the official Windows installer. Select the WSL 2 backend and Linux containers. Open Docker Desktop and wait until the engine is running. Its current WSL requirement is version 2.1.5 or later. [S2]

```
docker version
docker compose version
docker info --format "{{.OSType}}"
```

☐ 4. Confirm the readiness evidence

Docker version must include both Client and Server. The operating-system check above must return linux. Keep the outputs; do not share the private .env or any identity keys.

☐ 5. Protect existing test data

Keep a private backup of .env and the local data directory. Decide which sample records matter before moving to Docker databases. Do not delete volumes or regenerate identities over an existing network.

☐ 6. Return the setup results

Provide the WSL version, Docker Client/Server result, Compose version and any error text. The transition can then be validated on actual containers rather than inferred from source files.

03 / CONFIRMED SOFTWARE GAPS

Fix these before switching modes

Priority P0 blocks trustworthy blockchain results. Priority P1 blocks a reliable transition or its evidence.

P0 - An old simulated receipt can be relabeled as real

The registration duplicate path returns stored proof data while replacing its provider and simulated flag with the currently selected adapter. An isolated in-memory reproduction returned **hyperledger-fabric** and **simulated: false** for an old simulated transaction, with **zero ledger writes**. No project data was used or modified. [R4]

```
Stored transaction:  simulated-tx-assessment
Returned provider:   hyperledger-fabric
Returned simulated:  false
Actual ledger writes: 0
```

Required fix: retain immutable ledger provenance, prevent cross-ledger receipt/idempotency reuse and verify the matching Fabric state before acknowledging a prior registration. Archive simulated proof history or replay approved test data as explicitly new Fabric transactions; never just rename receipts.

P0 - Normal startup still selects the simulator

Confirmed: scripts/dev-local.mjs hardcodes LEDGER_DRIVER=file after reading .env. Base Docker Compose and Verification configuration also default to file. Stop-Fabric.ps1 recreates Verification on the file ledger. Changing only .env is insufficient. [R1, R2]

Required fix: choose Fabric for the normal test-system run, reject an unavailable Fabric dependency clearly, and make stopping the network preserve an unavailable state. Keep file adapters only for explicit unit-test fixtures. The Fabric adapter itself currently does not catch a failed Fabric write and switch to file.

P1 - Readiness confuses the database with the blockchain

Verification /ready combines ledger.simulated with metadata-store.simulated. A second isolated reproduction returned **ledgerDriver: fabric** and **simulated: true** when only the off-chain metadata store was file-based. Report ledger and database readiness separately. [R5]

P1 - Request budgets are too short for slow commits

Records defaults to a 5-second publication timeout; the Fabric adapter permits 15 seconds for endorsement and 60 seconds for commit status. A slow real commit can outlive the HTTP request. Align timeouts or use durable job polling, then test a commit that succeeds after the caller times out. A transaction failure from this mismatch has not yet been observed on live Fabric. [R6]

Harden the existing bootstrap

These are source-based findings and verification gaps. Container-specific failures remain unproven until Docker is available.

1. Prove startup success, rather than printing it

Start-Fabric.ps1 checks command exit codes but ends by asking the operator to inspect /ready. Add bounded health/readiness waits and a synthetic commit/read smoke test. In bootstrap-channel.sh the orderer wait loop breaks after its retry limit instead of failing at that point. Report the failed dependency precisely. [R2, R7]

2. Make restarts and chaincode upgrades predictable

Artifact generation rewrites the channel block on each run. Chaincode packaging reuses labrecord-contract_0.1.tgz whenever it exists, even if source files changed; approvals are issued again at sequence 1. Preserve network artifacts, compare source/package identities, check existing approvals and verify both organizations before committing. Provide a separate upgrade path rather than silently retaining stale code. [R7, S3]

3. Verify runtime, dependency and identity compatibility

Gateway/gRPC packages are optional dependencies even though Fabric mode requires them. Chaincode exports an ES module, while the installed Fabric loader uses require(). This is a runtime-version compatibility risk, not a demonstrated failure on Node 24. Pin/test the actual chaincode runtime and package. Also verify the generated private key is readable by the Verification container running as node: generation applies mode 0600. [R8, S5]

4. Choose and test one network layout

The existing Fabric overlay is intended for the Docker application: peer DNS names and operations URLs resolve on fabric_net. Native Windows services cannot use those internal names directly. The simplest initial integration is the existing full Docker layout. A lighter native-app option would need explicit loopback port mappings, host certificate paths and tested endpoint configuration. Do not assume either layout is proven. [R2, R9]

5. Make blockchain test results measurable

The Nodes page already checks each node health endpoint plus one ledger query. It does not check each peer's channel membership, block height/hash, installed contract version or synchronization lag. Add those measurements and real commit evidence. A healthy /healthz response alone is not proof of endorsement, commit or peer agreement. [R9, S6]

Policy correction: the current channel requires MAJORITY Endorsement across two organizations, so both are needed for normal new writes. The existing peer-recovery instructions suggest writing while the verifier peer is down; that expectation must change. Under the present policy, such writes should fail or remain pending. [R3, R10, S3]

05 / IMPLEMENTATION SEQUENCE

Transition in controlled stages

Owner: Codex for repository changes; you for machine setup and any hardware or data-retention decisions.

STAGE / OWNER	WORK	EXIT EVIDENCE
A - You	Install WSL/Docker, verify Linux engine, free working memory and preserve existing data.	Prerequisite commands pass.
B - Codex	Correct ledger defaults, shutdown behavior, receipt provenance, metadata labeling and timeout/retry handling.	Regression tests reject simulated receipts in Fabric mode.
C - Codex	Harden artifact reuse, package/version checks, identity access and readiness waits. Keep generated keys out of the Docker build context.	First start and repeated start both pass without resetting history.
D - Codex + you	Bring up peers/orderer, join labrecords and install/approve/commit the contract for both organizations.	Network-specific identity, package ID, sequence and policy captured.
E - Codex	Register a fresh synthetic proof; read it independently from both peers; run the complete laboratory release/QR flow.	Committed transaction ID and matching proof/history on both peers.
F - Codex + you	Run failure/retry, restart and backup/recovery checks. Review the acceptance checklist on page 6.	Evidence bundle with Pass/Fail/Not Run for every case.

Data migration decision - preserve provenance

Recommended first run: archive the old simulated verification data and create fresh synthetic cases on Fabric. Keep accounts and sample clinical records where feasible, but do not carry a simulated release receipt forward as a Fabric proof. Moving the full application to Docker/MySQL also needs an explicit local-data import or a documented fresh test dataset. [R4]

If old cases must be retained as blockchain demonstrations, implement a reviewed replay workflow: register versions in sequence, create new actual transaction IDs and record the replay time. Preserve original approval timestamps as record data, not as claims about when the blockchain transaction occurred.

Scope of the first milestone

Two peers and one orderer are enough for a real local permissioned-blockchain experiment. A Fabric CA enrollment service, public cryptocurrency, mining and public hosting are not prerequisites for this milestone. Add independent hosts and at least three Raft orderers only when the research scope includes host independence or orderer failover; the current one-orderer design cannot demonstrate that fault tolerance. [R3, S4]

06 / ACCEPTANCE CHECKLIST

Evidence that the blockchain is real

Owner: Codex runs the tests with you. All real-network cases below are currently NOT RUN.

ID	TEST AND EXPECTED RESULT	EVIDENCE TO KEEP
T1	Network identity: both peers join labrecords; contract version/sequence and endorsement policy agree.	Channel info, MSP IDs and committed definition.
T2	Commit: register a new synthetic hash. Return success only after a valid commit; read the same hash/version from both peers.	Transaction ID, commit status, peer query outputs.
T3	Full workflow: approval -> storage -> Fabric registration -> release -> patient/QR verification. Ledger fields remain free of clinical details.	Record version, ciphertext hash and redacted QR response.
T4	Idempotency/provenance: retry the same request, including after a simulated caller timeout. No duplicate version; old file receipts are rejected or explicitly migrated.	Request key, receipt provenance and peer history.
T5	Peer loss: stop the verifier peer. New writes must not report success under the two-org policy; restore it and retry safely.	Failed/pending write, then actual committed retry.
T6	Orderer loss: stop the sole orderer. New writes stop; restart it and recover. Do not describe this as automatic orderer failover.	Node state, write failure/recovery and transaction IDs.
T7	Persistence: restart containers without deleting volumes or identities. Existing proofs, hashes and history remain readable on both peers.	Before/after peer state and heights.
T8	Tamper/revoke: verify a wrong hash fails, a revoked proof no longer verifies, and earlier history remains available.	Verification results and ledger history.
T9	Monitoring/recovery: node status follows failures; compare peer heights and hashes after recovery. Restore to an isolated target with matching identities and data.	Timestamped health, block evidence and recovery report.

Completion rule: configuration files, a green node badge, a transaction-shaped string or passing mock tests are insufficient. The milestone is complete only when an actual committed transaction is independently visible on both peers and the application uses it. [R6, R9, S3]

For every test, record: **Pass / Fail / Not Run**, execution date, source/package version, transaction or record ID, expected result, actual result and evidence location.

Reviewer: _____ Test run date: _____

07 / REPOSITORY EVIDENCE

Where the findings came from

Paths are relative to Laboratory_blockchain. Line numbers identify the source reviewed on 08 September 2026.

R1 Ledger defaults

scripts/dev-local.mjs:24-32 hardcodes file; services/verification-service/src/config.js:9 defaults to file; docker-compose.yml:121 selects file. Only driver settings were read from .env; no secrets are included here.

R2 Start/stop and deployment topology

blockchain/fabric-network/Start-Fabric.ps1:13-29 generates, joins and starts the overlay; Stop-Fabric.ps1:9-13 recreates the file adapter. docker-compose.fabric.yml defines internal peer/orderer networks and Fabric Verification settings.

R3 Actual chaincode and endorsement configuration

blockchain/chaincode/src/index.js and src/labrecord-contract-core.js contain register/version/query/verify/release/revoke methods. blockchain/fabric-network/config/configtx.yaml:50-51 sets MAJORITY Endorsement; lines 55-63 define etcdraft with one consenter.

R4 Receipt provenance defect

services/verification-service/src/services/verification-service.js:72-135 reuses cached operations and relabels duplicate registrations. A synthetic in-memory reproduction returned a simulated transaction as hyperledger-fabric, simulated=false, with zero blockchain writes. MySQL registration rows also lack their own provider flag; provenance must be reconciled with stored transactions.

R5 Readiness labeling defect

services/verification-service/src/app.js:81,92 combines metadata simulation and ledger simulation. Isolated HTTP reproduction: status=200, ledgerDriver=fabric, top-level simulated=true, dependency ledger simulated=false.

R6 Commit handling and mismatched request budgets

services/verification-service/src/adapters/fabric-ledger.js:60-77 defines deadlines and checks commit status. services/records-service/src/config.js:16 defaults CLIENT_TIMEOUT_MS to 5000; src/clients/publicationClients.js aborts the outgoing call on that timer.

R7 Artifact and lifecycle handling

blockchain/fabric-network/scripts/generate-artifacts.sh:15-28 recreates the block and copies keys. scripts/bootstrap-channel.sh:49 breaks the orderer wait; lines 75-107 reuse a fixed package, approve sequence 1 and query commitment.

R8 Runtime and private material

services/verification-service/package.json:26 lists Gateway/gRPC as optional; Dockerfile:18 runs as node. blockchain/chaincode/package.json:6 selects module; installed fabric-shim/lib/contract-spi/bootstrap.js:77 uses require(). .dockerignore does not exclude blockchain/fabric-network/generated/.

R9 Node monitoring

services/verification-service/src/services/node-status.js calls fixed node /healthz URLs and ledger.health(); apps/web/src/pages/AdminPages.jsx displays those results. No per-peer height or block-hash measurement exists in that monitoring path.

R10 Tests and documentation gap

blockchain/chaincode/test/labrecord-contract.test.js uses MemoryStub. Verification tests use file adapters/mocks. tests/e2e/specs/full-laboratory-flow.spec.js skips without E2E_BASE_URL and does not require Fabric mode. docs/BLOCKCHAIN.md peer-recovery steps conflict with the two-org write policy.

08 / REFERENCES AND HANDOFF

Use this as the transition worklist

Official references checked on 08 September 2026. External documentation explains platform behavior; repository findings are supported by page 7.

S1 Microsoft - Install WSL

Administrator installation, reboot, distribution setup and checking WSL versions.

<https://learn.microsoft.com/en-us/windows/wsl/install>

S2 Docker - Install Docker Desktop on Windows

Supported Windows configurations, WSL backend, virtualization and minimum memory.

<https://docs.docker.com/desktop/setup/install/windows-install/>

S3 Hyperledger Fabric 2.5 - Chaincode lifecycle

Organization approvals, committed definitions and endorsement policy.

https://hyperledger-fabric.readthedocs.io/en/release-2.5/chaincode_lifecycle.html

S4 Hyperledger Fabric 2.5 - Ordering service

Raft quorum and the limitations of a single ordering node.

https://hyperledger-fabric.readthedocs.io/en/release-2.5/orderer/ordering_service.html

S5 Hyperledger - Node chaincode compatibility

Contract library/runtime compatibility and chaincode container packaging.

<https://github.com/hyperledger/fabric-chaincode-node/blob/main/COMPATIBILITY.md>

S6 Hyperledger Fabric 2.5 - Operations service

Health probes, metrics and operations endpoint behavior.

https://hyperledger-fabric.readthedocs.io/en/release-2.5/operations_service.html

Assessment checks completed

Contract suite: **5/5 passed**. Verification suite: **11/11 passed**. Static infrastructure validator: **passed**. Receipt-provenance and readiness-label defects: **reproduced using isolated fixtures**. Fabric deployment, real transaction commits, per-peer agreement, failure recovery and full real-network E2E: **Not Run**.

Handoff

You can work on: the six setup items on page 2 and the decision about preserving old test data.

Codex can start next: the code corrections on page 3 and bootstrap changes on page 4, followed by deployment once Docker/WSL is ready.

No migration or blockchain configuration changes were applied during this assessment. The earlier admin UI changes remain in place. This report does not certify production suitability or completion of the transition.